Assignment 01: Building a ReAct Agent from Scratch

仿 CS336 作业体例·DeepSeek / Ollama ·HotpotQA

Independent Study Assignment

Version 2.0.0 | 23 October 2026

1 Assignment Overview

这是一份**实现型**作业。你将自己写一个 ReAct（Reason + Act）Agent，从一个纯文本语言模型出发，构建**模型适配器、动作协议、解析器、工具系统、控制循环、观测历史、轨迹记录、评测器**，最终在 **HotpotQA** 的真实公开数据上测得自己的分数。项目**不训练 LLM 权重**；重点是理解 *inference-time agent system* 的工作机制。

本作业的出题方式参考你提供的 *CS336 Assignment 1 (basics), Version 26.0.3*：先给必要的背景解释，再用 **Problem / Deliverable / Tests / Hint / Points** 精确定义需要你实现的功能。与之相同，Starter 中**没有 Agent 的实现**，只有测试、配置样例、空白包及少量 `adapters.py` 函数签名。

你将实现（What you will implement）

1. 统一 LLM 接口（**DeepSeek API** 和 **本地 Ollama `qwen2:7b`** 两种运行模式）。
2. 可验证的 Action / Final 文本协议及其解析器。
3. 工具注册、参数验证、错误隔离、`search` / `lookup`。
4. 完整的 ReAct Agent Loop、停止条件、超时及上下文管理。
5. 运行轨迹 JSONL、可重现的测试和实验脚本。
6. HotpotQA distractor 环境与答案 / supporting facts 评分。

你将运行（What you will run）

1. 不联网的 `MockLLM` 组件测试和端到端测试。
2. 在 DeepSeek 或 Ollama 中任选一种完成真实连通性测试；推荐有条件时都测。
3. 固定 20 题 Smoke、50 题 Dev、200 题 Eval 三个**互不重叠**的 HotpotQA 子集。
4. Direct Answer、Single Retrieval、ReAct 三个系统的对照实验。
5. 至少一个消融和五个错误案例分析，生成实验报告。

时间预算：建议 2–3 周，约 25–40 小时。**总分** 100 分，选做最高 20 分。分数是用来检查工作完整性的，不设性能 “及格线”。

> **课程精神：**可以查官方 API 文档、学习概念、用 AI 讨论接口和 Debug 思路；建议核心代码自己手写，并关闭 AI 自动补全。**严禁**使用 LangChain、LangGraph、CrewAI、AutoGen 或现成 Agent Executor 替代循环；也不要复制现成 ReAct 教程的完整实现。

1.1 1.1 What you can use / What you cannot use

允许：Python 标准库、`httpx` / `requests` / `openai`（**仅作 HTTP / Chat API Transport**）、`pytest`、`python-dotenv`、`tqdm`、官方 HotpotQA 数据与官方评分脚本。允许调用托管模型或本地推理服务器；这是一个 *Agent* 系统实现 任务，不是从零训练 Qwen 的任务。

必须自己写：`parse_action`、`execute_tool`、ReAct Loop、Prompt、历史构造、检索规则、观测回填、trace、批量评测编排、分割与统计。`openai` SDK 不可以替你完成 Agent Loop。

不要求：手写 HTTP/TLS 栈、重训模型、写 Tokenizer、微调 Qwen、部署向量数据库或 GPU。不可使用 `eval`、`exec` 执行模型文本，不可把 API Key 写在代码或日志里。

1.2 1.2 How the code looks

```
react-assignment/
```

```
├── adapters.py              # 唯一固定的公共调用接口：只有签名和 NotImplementedError
├── react_agent/             # 你的实现目录：初始只有 __init__.py
│   └── __init__.py
├── tests/                   # 已写好的离线验收测试；不要修改
│   ├── conftest.py
│   ├── test_protocol.py
│   ├── test_tools.py
│   ├── test_loop.py
│   └── test_metrics.py
├── .env.example             # DeepSeek / Ollama 环境配置
├── pyproject.toml           # 环境依赖
├── .gitignore
├── README.md
├── data/README.md           # 真实数据需自行下载
└── reports/.gitkeep         # 你自己的答题报告
```

和 CS336 一样，**`adapters.py` 仅允许写薄薄的胶水代码**：它负责调用你在 `react_agent/` 中实现的函数；不能把真正的逻辑全部写进 `adapters.py` 来通过测试。测试不依赖网络，不需要 GPU 或 API Key。建议每完成一个小节就运行对应测试。

2 Background: What are you implementing?

2.1 2.1 LLM、Policy 与 Agent Controller

普通问答程序只运行一次 `LLM(question) → answer`。ReAct Agent 的关键是 **"模型提出动作，程序执行动作，环境返回观察，再由模型决定下一步"**。设历史 h_t 包含初始问题和已经产生的 Action / Observation，则

$$a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_t = T_{\text{tool}}(a_t), \qquad h_{t+1} = h_t \oplus (a_t, o_t).$$

这里 π_θ 表示调用 LLM 的条件决策分布，T_{tool} 是你实现的工具执行函数，$\oplus$ 表示把本轮可见动作和观察追加到历史。**本作业不对 θ 求梯度，也不要求使用 PPO 或训练 reward model**。 ReAct 和 RL 可以结合，但 ReAct 推理循环本身不等于强化学习训练。

一条成功的轨迹（仅为**格式示意**，不是解题代码）：

```
Question: Which country is the birthplace of the author of Book X?
LLM → {"type":"tool_call","tool":"search","args":{"query":"Book X author","k":3}}
Tool → [Document A, Document B, ...]
LLM → {"type":"tool_call","tool":"lookup","args":{"title":"Document A"}}
Tool → sentence 0: ...; sentence 1: ...
LLM → {"type":"final","answer":"Country Y","evidence":[["Document A",1]]}
```

原始 ReAct 常以 Thought / Action / Observation 交替表达；这次采用**可解析的外显 Action/Final JSON**。你可以允许模型输出一句简短可见的动作说明，但**不要索取或存储模型隐藏的 chain-of-thought**。实验关注工具使用和可观测控制流。

> **Problem (foundations): Agent 基础概念（8 points）**
>
> (a) 说明 Agent Controller 与 LLM 的责任边界。为什么仅靠 prompt 不能阻止一个工具无限循环?
>
> (b) 解释状态 h_t、动作 a_t、观察 o_t 分别由谁创建。为什么 Tool 返回值不能直接信任?
>
> (c) ReAct 中的交互轨迹与强化学习中的 trajectory 有何异同? 这次为什么不需要计算 policy

gradient?

(d) 如果 HotpotQA 的金标准 supporting facts 不小心进入 prompt，为什么 EM/F1 会失去可信度?

Deliverable: `reports/writeup.md` 中四段简答，各 100–200 字。

3 Environment and Model Backends

3.1 3.1 Install and connectivity

推荐 Python 3.11+，按 Starter 的 `pyproject.toml` 安装依赖。运行:

```
python -m venv .venv
source .venv/bin/activate                # Windows 使用 .venv\Scripts\activate
python -m pip install -e '.[dev]'
python -m pytest -q
```

首次运行测试预计**会失败**，这是预期行为，因为接口尚未实现。不要通过修改测试来消除失败；逐个实现对应功能。

本地模型的初始化（此命令来自 Ollama 的模型命名规则）:

```
ollama pull qwen2:7b
ollama serve                             # 如果服务已在后台启动则无需重复运行
ollama list
```

本地 Ollama OpenAI-compatible API: `http://localhost:11434/v1/chat/completions`。Qwen2 7B 的量化文件约为数 GB；运行时还需要额外内存与上下文缓存。没有独显也能尝试，但生成速度取决于机器。

DeepSeek API: `https://api.deepseek.com/chat/completions`，从 `DEEPSEEK_API_KEY` 读取密钥。**2026 年的官方文档默认示例模型为 `deepseek-flash`**（可选 `deepseek-v4-pro`）；不要将已经变动的旧 `deepseek-chat` 名称硬编码。模型名称必须可配置，以官方最新文档为准。

> **Backend configuration:** 两种模型都走 *OpenAI-compatible Chat Completions* 的共同子集，只依赖 `messages`、`model` 和返回的 assistant 文本。**不要**依赖某一家的 native function calling、`reasoning_content` 或特殊 tool schema 才能通过必做题。

推荐的环境变量如下（完整范例见 `.env.example`）:

变量	DeepSeek 示例	Ollama 示例
`REACT_BACKEND`	`deepseek`	`ollama`
`REACT_MODEL`	`deepseek-flash`	`qwen2:7b`
`REACT_BASE_URL`	`https://api.deepseek.com`	`http://localhost:11434/v1`
`DEEPSEEK_API_KEY`	在本机单独设置	不需要
`REACT_MAX_STEPS`	8	8
`REACT_MAX_TOOL_CALLS`	6	6

3.1.1 3.2 Model adapter contract

你需要实现一个统一的逻辑接口 `generate(messages) -> str`。消息为 `list[dict]`，每项至少有 `role` 与 `content`；返回只保留模型**用于下个动作的可见文字**。允许使用 `openai` SDK 帮你构造 HTTP 请求，但模型差异、异常处理和最终字符串提取由你实现。

请思考：HTTP 请求可以成功，但模型返回空字符串；可以收到限流 `429`；本地服务器可以不可达；模型还

可能输出不合法 JSON。**这些不是同一种失败**，不应统统写成 `except: pass`。

> **Problem (backend): Two model backends（12 points）**
>
> (a) 实现同一个 Chat API 适配层，使配置从 DeepSeek 切换为 Ollama 时**不需要修改 Parser、Tools 或 Agent Loop**。两种后端都要有配置入口，但没有付费 API Key 时允许只实际使用 Ollama。核心生成方法接受历史并返回可见文本。
>
> (b) 编写超时、连接错误、HTTP 4xx/5xx、空 response 的错误路径。对可重试的瞬态错误实现**有限次数**重试或明确返回受控错误；永久认证失败不应无限重试。
>
> (c) 在你的运行环境中，**任选一种后端**发起至少一次真实请求：DeepSeek 的 `deepseek-flash`（或当时官方支持的可配置模型），或者 Ollama 的 `qwen2:7b`。推荐有条件时对两者都测试。没有 DeepSeek API Key 不扣分，但必须保留其配置与实现入口，并清楚记录未测试的原因。
>
> **Deliverable:** `react_agent/` 中的实现，`reports/backend.md` 的一张表（模型 ID、base URL、是否测试、能否连通、实际调用耗时、异常说明）；未测试的一方标记为 `not tested`，不得虚构数据。**严禁提交真实 API Key**。
>
> **Tests:** `test_loop.py` 使用不联网的 Stub Model 验证组件接口。真实联网检查由你执行，不在 pytest 默认路径中调用付费 API。
>
> **Hint:** 同一条 user observation 可以作为下一个 `user` 消息追加到对话；不必使用带有 `tool_call_id` 的特定 API 原生工具消息。

4 Action Protocol and Parsing

4.1 4.1 A minimal textual protocol

规定：每一次 `generate` 返回恰好一个 JSON 对象。两种互斥结果：

```
{"type":"tool_call","tool":"search","args":{"query":"Ada Lovelace father","k":3}}
{"type":"final","answer":"Lord Byron","evidence":[["Ada Lovelace",2]]}
```

`type` 只能是 `tool_call` 或 `final`。工具调用需要 `tool`（非空字符串）与 `args`（对象）；最终答案需要 `answer`（字符串），`evidence` 为 `[title, sentence_id]` 列表，索引从零开始。可允许省略 `evidence` 但必须按空数组处理。额外输出的自然语言、Markdown 围栏或多对象 JSON 不属于本作业的**严格模式**；你可以在报告中讨论是否值得加入容错模式。

请不要把 `Final` 当成“要执行的工具”；它是一个停止事件。也不要让模型直接输出任意可执行 Python 代码。模型是不可信的行动提议者，Parser 和执行器才是决定能否执行的边界。

> **Problem (protocol): Parser and schema（14 points）**
>
> (a) 设计并实现 `parse_action(text: str) -> dict`，对于合法动作返回**经过验证的**字段，不合法输入应抛出 `ValueError`（或你的内部错误在 adapter 中统一转为 `ValueError`）。
>
> (b) 至少验证：不合法 JSON、顶层类型错误、未知 `type`、缺少 `tool/args/answer`、错误的 `args` 类型、错误的 evidence 结构、额外尾随非 JSON 文本。
>
> (c) 让模型输出与 Tool Registry 解耦：`parse_action` 不要求知道当前有哪些工具；工具名是否可用由执行器判断。
>
> **Deliverable:** parser 代码；`reports/writeup.md` 回答“为何要区分语法正确与工具可执行”。
>
> **Tests:** `python -m pytest tests/test_protocol.py -q`，不可通过弱化测试来满足要求。

5 Tools, Retrieval, and the Environment

5.1 5.1 A tool is not an arbitrary function call

工具执行器至少完成：**名字白名单 → 参数验证 → 调用 → 受控观察文本 / 错误 → 长度限制**。即使模型请求 `delete_all_files`，你的程序也不能通过反射到全局命名空间执行它。环境中的网页 / 文档是**数据**而非优先级更高的系统指令。

本次必做工具（具体检索算法由你写，不给实现）：

- `search(query: str, k: int = 3)`：从当前问题关联的候选文档集合中，返回最多 k 个按相关性排序的标题。最小版本可自己实现字词匹配 / BM25-lite；不得把正确 supporting facts 直接加权塞给检索器。
- `lookup(title: str)`：返回该标题文档的**全部句子**，并保留原来的从零开始的句子编号，以支持证据预测。

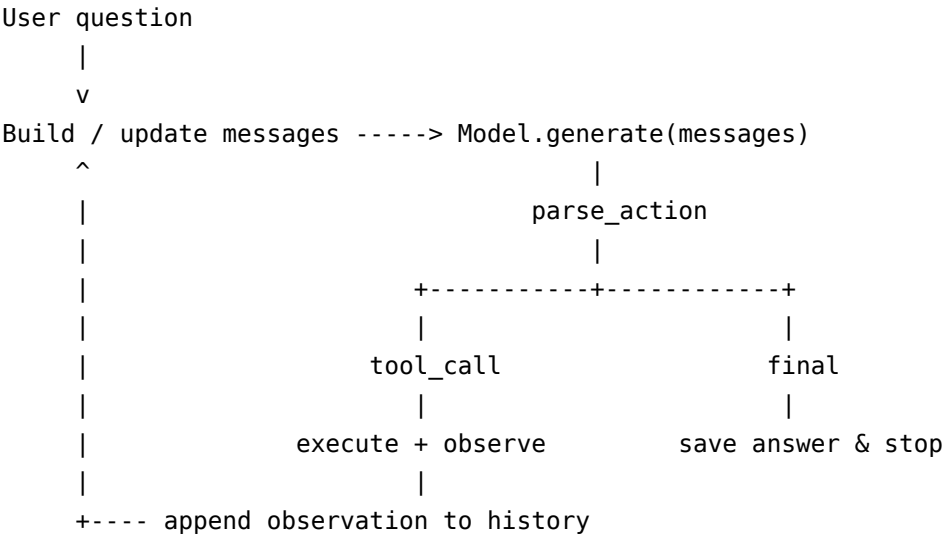
最初用几段自己编造的小文章运行工具；进入 HotpotQA 后，每道题都有单独的 candidate corpus。**一个问题的搜索结果不能泄露下一题的 context**。

> **Problem (tools): Safe executor（12 points）**
>
> (a) 实现 `execute_tool(action, tools) -> dict`:成功返回 `{"ok": true, "observation": "..."}`; 失败返回 `{"ok": false, "observation": "ERROR: ..."}`。调用永远只来自显式传入的 `tools` 字典。
>
> (b) 对未知工具、参数缺失、工具抛异常、观察结果过长等情况安全返回错误。失败计入一次工具调用，Agent 随后可以尝试修正参数。
>
> (c) 文档搜索至少支持 k 限制、确定性排序、无命中返回；`lookup` 保留 sentence ID，未知标题返回可恢复的错误。
>
> **Deliverable:** 工具代码与两条 “合法工具调用 / 恶意或无效工具调用” 的测试轨迹。
>
> **Tests:** `python -m pytest tests/test_tools.py -q`。

6 The ReAct Control Loop

6.1 6.1 State machine

你不需要框架；一个明确的状态机就足够：



关键是两个预算不要混淆：`max_steps` 限制模型决策次数，`max_tool_calls` 限制环境调用次数。**完成 Final 之前必须停止继续调用模型**。超预算后给出结构化的失败结果，不得假装成功。模型输出无效 JSON 时也应有可复现的策略（立即错误、一次修复提醒等，你需要说明你的选择）。

Problem (agent_loop): Implement the controller（18 points）

(a) 实现 `run_agent(question, model, tools, max_steps=8, max_tool_calls=6) -> dict`。返回至少包括 `status`、`answer`、`evidence`、`trace`、`num_tool_calls`。`status` 可取 `final`, `max_steps`, `tool_limit`, `error`。

(b) `model` 必须能通过一个 `generate(messages)` 方法被调用；测试将注入 **ScriptedModel**，按顺序给你动作，不需要联网。

(c) 每次 Tool 调用的 observation 必须进入**下一轮发送给模型的 messages**，不是只打印到终端。模型可以依赖第一次 lookup 的结果做第二次 search。

(d) 实现预算上限、终止条件、异常记录。不得在任一正常测试轨迹中多出额外 LLM 调用或重复执行工具。

Deliverable: 可复用 Agent Loop；两条端到端 Mock 轨迹：多跳成功、预算耗尽。
Tests: `python -m pytest tests/test_loop.py -q`。
Hint: 先用一个只会按顺序返回预先写好 JSON 的类测试控制流，再接 DeepSeek / Ollama。不要同时 Debug 网络、Parser 和循环。

7 Traces, Error Handling, and Reproducibility

7.1 7.1 A trace is your debugging dataset

每一步建议记录 `step_index`、输入消息长度、原始可见模型输出、解析后 Action、工具名、参数、Observation 摘要、错误类型、耗时、后端与模型名。最后追加终止状态。建议将**每道题一行 JSON** 写入 `.jsonl`，不要将 API Key / Token / 个人路径写进产物。大段 Observation 可另存文件，但需要能根据 trace 重建运行过程。

模型层级的 `usage` token 数可能并不是所有 Ollama / DeepSeek 返回中都存在，日志允许 `null`。**缺失不等于零**。如果日志中同时保存 prompt 与模型输出，可以计算平均调用次数、工具错误率以及按问题聚合的时延。

Problem (trace): Observability（8 points）

(a) 为每个样本生成具有稳定 `sample_id` 的 trace；能够仅凭 trace 解释这道题为什么失败。

(b) 对重复 `sample_id` 的断点续跑保持清晰策略，不应无意覆盖已有正确结果。

(c) 记录 `model`、`backend`、prompt 版本、检索方法、最大步骤数、工具预算、运行时间；预测与诊断信息分开存放。

Deliverable: `reports/traces/` 中至少三条脱敏的 JSONL 示例, `reports/writeup.md` 说明 trace schema。

8 Benchmark: HotpotQA

8.1 8.1 Which task are we actually evaluating?

必做：HotpotQA **distractor development split**。这是一个真实、公开、带有问题答案与 supporting facts 的多跳问答数据集。每条记录有 `_id`、`question`、`answer`、`supporting_facts` 以及 `context`；后者包含若干候选段落 `[title, sentences]`。某些候选段落与问题相关，其他是 distractors。

本次评测协议有意限定为「closed-candidate retrieval」：把该题 `context` 中的候选文档交给你写的 `search/lookup` 工具，但**绝不直接将全部候选文档塞进 Agent 的初始 prompt**。这能低成本检验多轮检索、上下文回填与停止逻辑，也适合本地 7B 模型。**这不是 HotpotQA fullwiki 排行榜设置**，不能将自己的成绩冒称为官方 fullwiki Leaderboard 结果。

下载与评测参考:

- 数据、格式与 `hotpot_evaluate_v1.py`：https://github.com/hotpotqa/hotpot
- 数据集主页：https://hotpotqa.github.io/
- 数据模式：`hotpot_dev_distractor_v1.json`；不要把含标准答案的文件直接交给模型作为完整 prompt。

重要：数据文件里的 `answer`、`supporting_facts`、`type`、`level` 在测试时是 **evaluator-only** 字段。模型可见的只有 `question` 及通过 `search` / `lookup` 取回的文本。`type` 和 `level` 可用于**离线分组分析**，不作为模型输入。

8.2 8.2 Official scores vs our teaching protocol

答案的标准评分采用**归一化的 Exact Match (EM) 与 token-level F1**。令预测 token 集合按计数计算交集，precision $P = c/m$，recall $R = c/n$，则:

$$F_1 = \frac{2PR}{P+R} \quad (P+R>0), \qquad \text{EM} = \mathbb{1}[N(\hat{y}) = N(y)],$$

其中 N 表示官方脚本的一致归一化操作（大小写、英文冠词、标点、空格）。`yes` / `no` / `noanswer` 需要按官方评分语义处理。Supporting-fact 预测使用 `(title, sentence_id)`，可计算 supporting-fact EM/F1 和 joint 指标。**与官方成绩比较时应直接复用官方评测脚本，不能只展示自行定义的宽松答案匹配**。

Problem (hotpot): Build the environment（12 points）

(a) 编写一个数据加载入口，从 HotpotQA JSON 解析 `context`，并为每道题建立**彼此独立**的可检索文档集合。

(b) 实现 `search` / `lookup` 适配接口：`run_search(context, query, k)` 返回标题列表；`run_lookup(context, title)` 返回指定标题及原句子序列。无命中、重复标题、大小写、空 query 都要合理处理。

(c) 初始 Prompt **只包含 question、工具说明和输出协议**；不允许包含 `answer`、`supporting_facts` 或完整 context。绝不读取别的题目的 gold label 来选文档。

(d) 以一种显式方式让模型在 Final 提交 `evidence`，句子编号保留原文编号；不能在最后利用金标准做 “证据修补”。

Deliverable: 热身的小规模运行日志与检索错误分析，报告中说明为什么该设置并非 fullwiki。
Tests: `python -m pytest tests/test_tools.py -q`。对真实 HotpotQA 的 end-to-end smoke test 需在你下载数据后自行执行。

9 Evaluation Protocol

9.1 9.1 Deterministic subsets

从官方 **dev distractor** 的全部 `_id` 出发，先按 `SHA256("42:" + id)` 的字符串十六进制值排序，再取 **20 / 50 / 200** 条作为三个互斥分组，分别命名 `smoke`, `dev`, `eval`。**不要**用 Python 内建 `hash()` 做排序，因为 Python 进程间 hash 随机化可能导致不同结果。提交三个 ID 列表的文件并固定内容；最终 `eval` 题目不参与 prompt / 检索参数调试。

阶段建议:

Split	N	用途	规则
Smoke	20	检查 API、格式与崩溃	可多次重跑
Dev	50	调 prompt、检索和预算	必须记录改动

Split	N	用途	规则
Eval	200	最终锁定系统的结果	参数锁定后一次完整运行

200 题的分数只代表**你锁定协议下的子样本结果**。若引用该分数，必须写明具体样本集大小与选取方式；不要称之为 “HotpotQA official leaderboard”。对于 Ollama 和 DeepSeek 应在**同一批 IDs 与候选文档**上比较，否则无法归因。

9.2 9.2 Metrics and official scorer

最终必须报告：`Answer EM`, `Answer F1`, `Supporting-fact EM/F1`（证据输出不完整也应诚实记分）、`Joint EM/F1`、`Average tool calls`, `Average LLM calls`, `Timeout / invalid-action rate`, `Wall-clock seconds / sample`。如果某运行没有提供证据预测，明确报告该限制而不是伪造 supporting-fact 分数。

官方脚本输入形状：

```
{
  "answer": {"sample_id_1": "predicted answer"},
  "sp": {"sample_id_1": [["Document Title", 1]]}
}
```

调用：

```
python hotpot_evaluate_v1.py predictions.json hotpot_dev_distractor_v1.json
```

注意： 上述命令会评估整个官方 dev 文件；如果 `predictions.json` 仅含 200 题，官方脚本会把其余题目视为缺失而拉低总体分数。必须为你选中的固定 IDs 生成**只含这 200 题的 gold 子集 JSON**，再将两个匹配的子集文件一起传给官方评分脚本；或者完整跑完官方 dev。提交所用脚本的来源和数据版本。不可将不同大小的预测集与 gold 集直接混比。

> **Problem (evaluation): Run a real benchmark（10 points）**
>
> (a) 实现稳定的数据切分，保存样本 ID 文件及摘要；确保 smoke/dev/eval 不重叠。
>
> (b) 运行固定 200 条 Eval，生成官方 scorer 可读取的预测文件和相应的 gold 子集。保留每条题目的 trace 与失败状态。
>
> (c) 使用官方评分脚本给出 Answer EM/F1、SP EM/F1、Joint EM/F1，同时报告准确的总题数、错误数与平均 LLM/Tool 调用数。
>
> **Deliverable:** `reports/metrics.json`、`reports/predictions.json`、`reports/eval_ids.txt`、`reports/writeup.pdf`。没有跑完时如实报告部分完成，**不可虚构任何分数**。
> **Tests:** `python -m pytest tests/test_metrics.py -q`，再做真实官方 scorer 交叉检查。

10 Experiments and Analysis

10.1 10.1 Three baselines on the same question set

1. **Direct**：不开放工具，让相同 LLM 直接回答问题。它不是作弊；它测量模型自身知道多少。
2. **Single Retrieval**：只允许一次 `search` + 一次 `lookup` 或一次预先定义好的检索流程，然后回答；不允许模型在收到 Observation 后再决定要查什么。清楚写下这条策略的具体实现。
3. **ReAct**：由模型在最多 8 个决策步、6 次工具调用的预算内交互式选择工具；记录完整 trace。

在**同一后端**内先比较这三个系统，再将整个实验矩阵在第二个后端上重复（如果 API 预算或本地速度允许）。温度、最大输出长度、观察截断阈值、检索后端、题目 IDs 需要控制；不同模型间的直接分数差距**不能只归因于 ReAct 机制**。

最少一个消融：例如 `max_tool_calls=1` vs 6、禁用 `lookup`、移除错误反馈、观察文本截断到不同长度、

固定检索 vs 多轮检索。先提出假设，再实验，最后解释成败。建议按照 `bridge` / `comparison`、工具调用次数或失败类型做细分分析，但不要在 Eval 后反复调参。

Problem (analysis): Empirical reasoning（6 points）

(a) 做出同一后端下 **Direct / Single Retrieval / ReAct** 的 EM/F1 对照表，至少包含一个你自行定义的消融实验。

(b) 抽样分析**至少五个失败 case**：是检索失误、缺少第二跳、工具参数出错、观察被截断、错误停止、答案归一化还是模型不服从动作协议?

(c) 回答 “什么情况下 Agent 的额外工具调用反而降低效果? ” 结合自己的 trace 举证；不要以性能变好作为必须成立的结论。

Deliverable: 4–6 页研究式报告（方法、实现、结果、消融、失败案例、局限与可复现性）。附运行环境、模型具体版本、后端、脚本命令、数据 ID、参数与真实耗时。

11 Bonus Challenges (up to +20 points)

Bonus A (+5): Native tool calling vs textual protocol. 同样的 Loop，替换 *Action Protocol* 为模型自身的函数调用接口；比较遵守格式率与 Tool success rate。注意 DeepSeek 与本地 Qwen2 API 的原生 tool calling 行为不一定一致。

Bonus B (+5): Context management. 自己实现滑动窗口、Observation 摘要或去重；检查是否牺牲证据定位能力，不得通过金标准挑内容。

Bonus C (+5): Stronger retrieval. 在同样的 distractor candidate corpus 内实现 BM25 / TF-IDF，与朴素词匹配检索比较；请注明是否安装第三方检索库以及是否自己写核心算法。

Bonus D (+5): ALFWorld / open retrieval. 将同一 Controller 迁移到 ALFWorld 或 HotpotQA fullwiki 设置；需要解释动作与观察协议差异、成功率或真实的 IR 基准评测方法。它们与本次 distractor 结果不可直接比较。

12 Submission and Grading

Problem	分值	必须提交
foundations	8	概念问答
backend	12	双后端配置 + 至少一种真实连通性证明
protocol	14	JSON Parser + 离线测试
tools	12	工具安全执行器 + 搜索与查阅
agent_loop	18	完整多跳循环 + Mock 测试
trace	8	可复现轨迹与配置
hotpot	12	基准环境 + 信息隔离
evaluation	10	真实 200 题固定子集 + 官方评分
analysis	6	对照 / 消融 / 错误分析
Total	**100**	**code.zip + writeup.pdf + outputs**

提交格式（模仿 CS336）：`code.zip` 包含**你自己实现的** `react_agent/`、`adapters.py`、运行脚本、配置和非密钥测试；`writeup.pdf` 包含上面各题的书面回答；`results/` 放 JSON 预测、锁定的 eval ID、度量、脱敏轨迹。**不要**将 `.env`、API Key、数 GB 的模型权重与原始 benchmark 全量数据放入提交压缩包。

验收顺序：`test_protocol` → `test_tools` → `test_loop` → `test_metrics` → 20 题 Smoke → 50 题 Dev → 200 题 Eval。通过 pytest 仅表示你的公共接口和几个行为案例满足要求；它**不证明**你的模型一定

有较高 EM/F1。

12.1 Reproducibility checklist

- ☐ 在没安装 Agent 框架且没配 API Key 的机器上，pytest 运行且 Mock tests 可通过。
- ☐ Backend 只切换配置即可替换 DeepSeek / Ollama；真实 API 调用成功或明确记录失败原因。
- ☐ 原始 `answer` / `supporting_facts` 未进入模型输入；单样本工具不跨题泄漏。
- ☐ `search`/`lookup` 的排序及 sentence ID 可复现。
- ☐ 20/50/200 三组 ID 固定且无交集；Eval 不被用来调 prompt。
- ☐ 预测和 gold 子集具有**完全一致**的 IDs。
- ☐ 官方 scorer、日志与实验报告中的成绩一致。
- ☐ 没有虚构费用、token 消耗、延迟或分数。

13 Low-Resource Tips and Debugging Strategy

> **Low-Resource Tip 1 — Zero-cost debugging.** 最先写一个模型替身，让它按顺序输出三段预定 JSON（search、lookup、final）。离线测试过后再调用真正的模型，节约 API 请求，也避免 Qwen2 生成速度成为初期阻碍。

> **Low-Resource Tip 2 — Debug the smallest dataset.** 先自己编两三篇短文，然后做 20 题 Smoke。只有出错率可控才进行 50 题 Dev 与最终 200 题 Eval；不得因为跑得慢而默默丢弃 “困难样本”。

> **Low-Resource Tip 3 — Local inference is not free of constraints.** `qwen2:7b` 虽然免托管 API 费用，但受 RAM/VRAM 与上下文长度限制。如果跑得太慢，先减少 Smoke 数量以调试，**不要改变最终声明的 Eval 集**，并记录实际模型量化与运行设备。

> **Low-Resource Tip 4 — Two models, one Agent.** 如果 DeepSeek 与 Qwen2 的 JSON 遵从性不同，先观察它们在相同 prompt 下输出了什么，再考虑可记录的格式重试策略；不能通过手动编辑失败答案获得更高分数。

14 Readings and Official Documentation

1. Yao et al., **ReAct: Synergizing Reasoning and Acting in Language Models**, arXiv:2210.03629. https://arxiv.org/abs/2210.03629
2. **Stanford CS336 Assignment 1 (basics)**, Version 26.0.3, **用户提供的 PDF**；本作业借鉴其 *problem / deliverable / tests / minimal starter* 体例，并非官方 Stanford 作业。
3. **DeepSeek API**：https://api-docs.deepseek.com/，检查当前模型 ID / Chat Completions 兼容性；不要依赖过期教程中的 `deepseek-chat` 名称。
4. **Ollama OpenAI compatibility**：https://github.com/ollama/ollama/blob/main/docs/api/openai-compatibility.mdx。
5. **Ollama qwen2:7b**：https://ollama.com/library/qwen2:7b。
6. **HotpotQA official site**：https://hotpotqa.github.io/；代码仓库和官方评测：https://github.com/hotpotqa/hotpot。

最后提醒：完整交付的含义是 自己实现了一个受控 *Agent* 系统，且报告真实实验结果。并不是 “把现成教程从 GitHub 复制下来再填一个 API Key”。